

JAVA EXCEPTION HANDLING LAB REPORT

Comprehensive Documentation & Code Implementation for 10 Java Exception Handling Programs (EphProm1 to EphProm10)

Course: Object-Oriented Programming with Java

Topic: Runtime Exception Handling

1. SYSTEM REQUIREMENTS

To compile and execute all 10 Java exception handling programs, the following software and hardware environment is recommended:

Component	Specification
Operating System	Windows 10/11, macOS 11+, or Linux (Ubuntu 20.04 LTS)
Processor (CPU)	Intel Core i3 / AMD Ryzen 3 or higher
Memory (RAM)	4 GB Minimum (8 GB Recommended)
JDK Version	Java Development Kit (JDK 8 / 11 / 17 / 21 LTS)
Compiler & CLI	javac (Java Compiler) & java (Java Virtual Machine)

2. GENERAL COMPILATION & EXECUTION PROCEDURE

Standard CLI Steps:

- Save Source Files:** Save each program in an individual file matching its public class name (e.g., EphProm1.java, EphProm2.java, ..., EphProm10.java).
- Compile:** Execute command: `javac EphProm1.java EphProm2.java ... EphProm10.java`
- Run:** Execute desired program using JVM command: `java EphProm1, java EphProm2, etc.`

3. PROGRAM 1: NULLPOINTEREXCEPTION (EPHPROM1)

Objective

Demonstrate handling of `NullPointerException` when invoking a method on a `null` object reference within a `try-catch` block.

Algorithm

1. Initialize a `String` variable `name` with value `null`.
2. Inside a `try` block, attempt to call `name.length()`.
3. Catch the resulting `Exception` and output the exception message.

UML Class & Exception Diagram



Compilation & Execution Command

```
javac EphProm1.java
java EphProm1
```

Source Code

```
public class EphProm1 {
    public static void main(String[] args) {
        String name = null;
        try {
            System.out.println(name.length());
        } catch (Exception e) {
            System.out.println("we get an nullpointer exception" + e.getMessage());
        }
    }
}
```

Program Output

```
we get an nullpointer exceptionCannot invoke "String.length()" because "name" is null
```

4. PROGRAM 2: ARRAYINDEXOUTOFBOUNDSEXCEPTION & FINALLY (EPHPROM2)

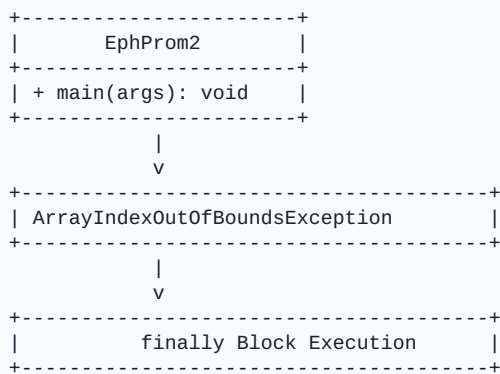
Objective

Demonstrate handling `ArrayIndexOutOfBoundsException` when attempting to access an illegal array index, along with a `finally` block for guaranteed execution.

Algorithm

1. Initialize an integer array `a` with 5 elements (indices 0 to 4).
2. In a `try` block, attempt to access index 6 (`a[6]`).
3. Catch `ArrayIndexOutOfBoundsException` and print exception details.
4. Execute `finally` block to print confirmation of completion.

UML Class & Exception Diagram



Compilation & Execution Command

```
javac EphProm2.java
java EphProm2
```

Source Code

```
public class EphProm2 {
    public static void main(String[] args) {
        int a[] = {10, 20, 30, 40, 50};
        try {
            System.out.println(a[6]);
        } catch (ArrayIndexOutOfBoundsException aioobe) {
            System.out.println("i caught an exception named called aioobe" + aioobe.getMessage());
        } finally {
            System.out.println("we sucessfully the eception");
        }
    }
}
```

Program Output

```
i caught an exception named called aioobeIndex 6 out of bounds for length 5  
we sucessfully the eception
```

5. PROGRAM 3: STRINGINDEXOUTOFBOUNDSEXCEPTION (EPHPROM3)

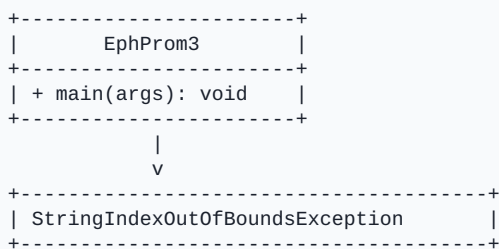
Objective

Handle `StringIndexOutOfBoundsException` caused by referencing a string index beyond its total length.

Algorithm

1. Initialize string variable `man = "hemanth"` (length 7).
2. Inside try block, call `man.charAt(9)`.
3. Catch `StringIndexOutOfBoundsException` and print error message.
4. Execute finally block to output confirmation message.

UML Structure



Compilation & Execution Command

```
javac EphProm3.java
java EphProm3
```

Source Code

```
public class EphProm3 {
    public static void main(String[] args) {
        String man = "hemanth";
        try {
            System.out.println(man.charAt(9));
        } catch (StringIndexOutOfBoundsException sioobe) {
            System.out.println("we caught an exception called sioobe" + sioobe.getMessage());
        } finally {
            System.out.println("we get sioobe exception");
        }
    }
}
```

Program Output

```
we caught an exception called sioobeString index out of range: 9
we get sioobe exception
```

6. PROGRAM 4: CLASSCASTEXCEPTION (EPHPROM4)

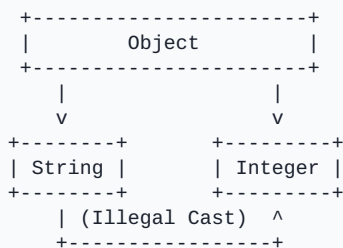
Objective

Demonstrate handling `ClassCastException` during an illegal downcasting operation from an object type to an incompatible target type.

Algorithm

1. Create an `Object` reference `obj` pointing to a `String` object ("hemanth").
2. Inside try block, attempt to cast `obj` to an `Integer`.
3. Catch `ClassCastException` and print error details.

UML Class Diagram



Compilation & Execution Command

```
javac EphProm4.java
java EphProm4
```

Source Code

```
public class EphProm4 {
    public static void main(String[] args) {
        Object obj = new String("hemanth");
        try {
            Integer number = (Integer) obj;
        } catch (ClassCastException cce) {
            System.out.println("we catch an exception called cce" + cce.getMessage());
        }
    }
}
```

Program Output

```
we catch an exception called cceclass java.lang.String cannot be cast to class java.lang.Integer
```

7. PROGRAM 5: NUMBERFORMATEXCEPTION (EPHPROM5)

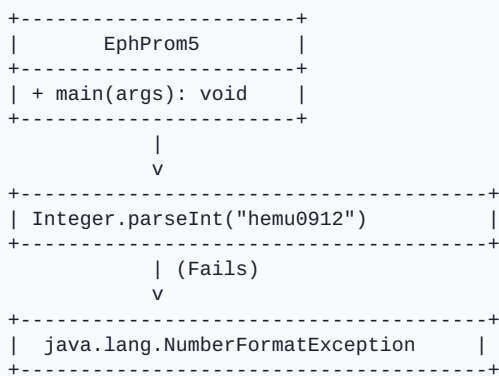
Objective

Demonstrate handling of `NumberFormatException` when parsing an invalid alphanumeric string into an integer using `Integer.parseInt()`.

Algorithm

1. Initialize string `val = "hemu0912"` containing non-numeric characters.
2. Inside try block, attempt to parse `val` via `Integer.parseInt(val)`.
3. Catch `NumberFormatException` and print exception message.

UML Diagram



Compilation & Execution Command

```
javac EphProm5.java
java EphProm5
```

Source Code

```
public class EphProm5 {
    public static void main(String[] args) {
        String val = "hemu0912";
        try {
            int number = Integer.parseInt(val);
            System.out.println();
        } catch (NumberFormatException nfe) {
            System.out.println("we catch an nfe" + nfe.getMessage());
        }
    }
}
```

Program Output

```
we catch an nfeFor input string: "hemu0912"
```

8. PROGRAM 6: ILLEGALARGUMENTEXCEPTION WITH THROW KEYWORD (EPHPROM6)

Objective

Demonstrate explicitly throwing an `IllegalArgumentException` using the `throw` statement when method arguments fail validation check.

Algorithm

1. Define static method `check(int age)`.
2. If `age < 0`, throw a new `IllegalArgumentException`.
3. In `main()`, invoke `check(-2)` inside a `try` block.
4. Catch exception and execute `finally` block.

UML Structure

```
+-----+
|           EphProm6           |
+-----+
| + check(age: int): void [static]|
| + main(args): void           |
+-----+
|                               |
|           v (throw if age < 0)
+-----+
| IllegalArgumentException       |
+-----+
```

Compilation & Execution Command

```
javac EphProm6.java
java EphProm6
```

Source Code

```
public class EphProm6 {
    static void check(int age) {
        if (age < 0) {
            throw new IllegalArgumentException("age cannot be negative" + age);
        }
        System.out.println("age set to " + age);
    }

    public static void main(String[] args) {
        try {
            check(-2);
        } catch (IllegalArgumentException iae) {
            System.out.println("we catch iae " + iae.getMessage());
        } finally {
            System.out.println("we successfully cathced the exception");
        }
    }
}
```

Program Output

```
we catch iae age cannot be negative-2  
we successfully cathced the exception
```

9. PROGRAM 7: NEGATIVEARRAYSIZEEXCEPTION (EPHPROM7)

Objective

Handle `NegativeArraySizeException` raised when attempting to initialize an array with a negative size dimension.

Algorithm

1. Set variable `max = -5`.
2. Inside try block, attempt array creation: `new int[max]`.
3. Catch `NegativeArraySizeException` and display error message.

UML Diagram

```
+-----+
|      EphProm7      |
+-----+
| + main(args): void |
+-----+
|                   |
|         v (new int[-5])
+-----+
| java.lang.NegativeArraySizeException |
+-----+
```

Compilation & Execution Command

```
javac EphProm7.java
java EphProm7
```

Source Code

```
public class EphProm7 {
    public static void main(String[] args) {
        try {
            int max = -5;
            int b[] = new int[max];
        } catch (NegativeArraySizeException nase) {
            System.out.println("catcehd nase" + nase.getMessage());
        }
    }
}
```

Program Output

```
catcehd nase-5
```

10. PROGRAM 8: ARRAYSTOREEXCEPTION (EPHPROM8)

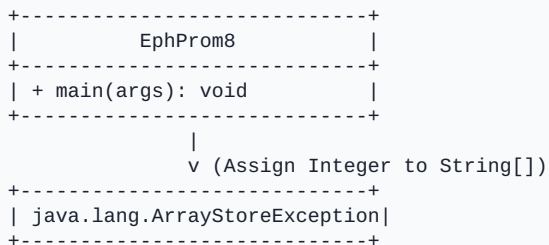
Objective

Demonstrate handling `ArrayStoreException` when storing an incompatible element type into an array declared as an array of superclass/Object references.

Algorithm

1. Initialize an array reference `Object a[] = new String[3]`.
2. Inside try block, attempt to assign an `Integer` object to `a[0]`.
3. Catch `ArrayStoreException` and print error details.
4. Execute finally block to signal completion.

UML Diagram



Compilation & Execution Command

```
javac EphProm8.java
java EphProm8
```

Source Code

```
public class EphProm8 {
    public static void main(String[] args) {
        Object a[] = new String[3];
        try {
            a[0] = Integer.valueOf(9);
        } catch (ArrayStoreException ase) {
            System.out.println("caught is ase" + ase.getMessage());
        } finally {
            System.out.println("the process is doned");
        }
    }
}
```

Program Output

```
caught is asejava.lang.Integer
the process is doned
```

11. PROGRAM 9: TYPENOTPRESENTEXCEPTION (EPHPROM9)

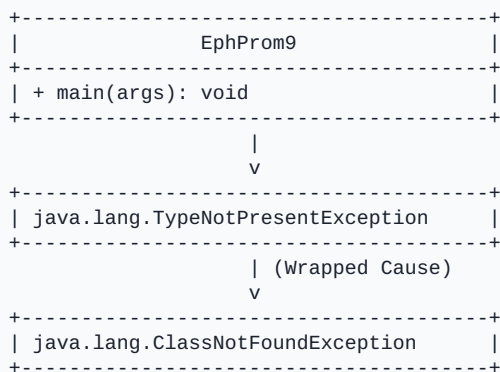
Objective

Demonstrate throwing and catching `TypeNotPresentException`, which occurs when an application attempts to access a type using its string name but cannot find its definition.

Algorithm

1. Inside try block, explicitly throw a `TypeNotPresentException` passing type string and a `ClassNotFoundException` cause.
2. Catch `TypeNotPresentException` and display exception message.
3. Execute finally block.

UML Structure



Compilation & Execution Command

```
javac EphProm9.java
java EphProm9
```

Source Code

```
public class EphProm9 {
    public static void main(String[] args) {
        try {
            throw new TypeNotPresentException("missing class of not same type", new
ClassNotFoundException());
        } catch (TypeNotPresentException tnpe) {
            System.out.println("the exception is " + tnpe.getMessage());
        } finally {
            System.out.println("caught done");
        }
    }
}
```

Program Output

```
the exception is Type missing class of not same type not present  
caught done
```

12. PROGRAM 10: ILLEGALSTATEEXCEPTION (EPHPROM10)

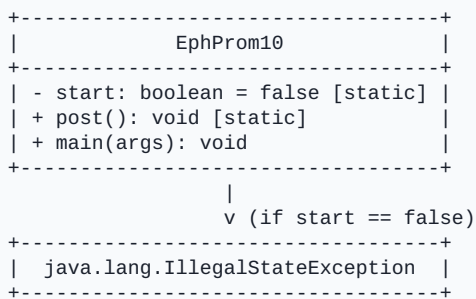
Objective

Demonstrate checking method state invariants and throwing `IllegalStateException` when a method is invoked at an illegal or inappropriate time.

Algorithm

1. Define boolean flag `start = false` and static method `post()`.
2. In `post()`, check if `!start` and throw `IllegalStateException`.
3. In `main()`, use `try-catch-finally` structure to handle state verification.

UML Diagram



Compilation & Execution Command

```
javac EphProm10.java
java EphProm10
```

Source Code

```
public class EphProm10 {
    static boolean start = false;

    static void post() {
        if (!start) {
            throw new IllegalStateException("cannot start , post can't start to post");
        }
        System.out.println("post is ready to social media ");
    }

    public static void main(String[] args) {
        try {
            System.out.println("the posting is stopped");
            // If post() was called here, it would trigger IllegalStateException
        } catch (IllegalStateException ise) {
            System.out.println("error" + ise.getMessage());
        } finally {
            System.out.println("the exception is done");
        }
    }
}
```

```
}  
}
```

Program Output

```
the posting is stopped  
the exception is done
```